Как компьютер работает с текстом

Что сохранить, когда превращаешь сообщение в данные

Одно сообщение — несколько возможных задач

Слайд 1. Как компьютер работает с текстом

В электронную приёмную пришло сообщение: «Я не могу войти в личный кабинет». Сотрудник почти сразу понимает, о чём речь. Он может переслать письмо в техническую поддержку, уточнить название сервиса или предложить следующий шаг. А что именно мы хотим поручить программе?

Первый шаг в работе с NLP — договориться о результате. NLP, natural language processing, — обработка естественного языка: методы, с помощью которых программы работают с человеческой речью и текстом. Мы будем заниматься письменным текстом. Нас интересует, как превратить его в подходящий для вычислений вид и как связать полученные числа с полезным действием.

Слайд 2. Сначала определим результат

Один вариант — выбрать отдел. Тогда на входе будет текст, а на выходе одна категория: «ИТ», «библиотека», «общежитие» и так далее. Другой вариант — найти в сообщении полезные сведения: название сервиса, время сбоя, код ошибки. Третий — подготовить ответ студенту. Сообщение одно, но результат работы системы в этих случаях устроен по-разному.

Начнём с выбора отдела. Допустим, после обработки сообщения программа возвращает строку `it`. Сотрудник видит письмо в очереди технической поддержки и отвечает на него сам. Для такой системы можно собрать набор обращений с известными категориями, сравнить предсказания с разметкой и посчитать ошибки. Позже мы именно так построим первый классификатор.

Теперь представим, что от программы ждут ответ: «Обратитесь в библиотеку и смените читательский пароль». Предложение выглядит связным, однако мы не знаем, о каком кабинете говорил студент. Программа добавила сведения, которых в сообщении не было. Чтобы отвечать по существу, ей может понадобиться база инструкций, уточняющий вопрос или возможность передать обращение человеку. В нашем примере можно начать с вопроса «Какой кабинет вы используете?». Он показан рядом с другими возможными результатами: категорией «ИТ» и извлечённым названием «личный кабинет». Задача стала шире, чем выбор категории.

Возьмём ещё одно обращение: «В комнате сломался замок, подскажите, куда написать». Для маршрутизации достаточно понять, что речь об общежитии. Для заполнения заявки не хватает номера комнаты. Для полноценного ответа нужны правила подачи заявки. Прежде чем обсуждать модель, полезно выписать три вещи: что подаётся на вход, что ожидается на выходе и по какому признаку мы узнаем, что задача решена.

Попробуем сделать это для трёх систем: фильтра нежелательной почты, поиска по библиотечному каталогу и помощника, который отвечает на вопросы о пересдачах. У первой результатом будет категория письма. У второй — список документов в порядке полезности. У третьей — ответ, который должен соответствовать вопросу и опираться на действующие правила. Даже если внутри всех трёх используются похожие представления текста, оценивать их одинаковым способом неудобно.

Как устроен ответ системы

Слайд 3. У NLP-задач разные выходы

В классификации мы выбираем из заранее заданных вариантов. Например, обращение относится к библиотеке или к технической поддержке. Сам текст может быть длинным, коротким, вежливым или раздражённым; ответ классификатора всё равно имеет одну и ту же форму. Это полезное упрощение: вместо свободного понимания всего сообщения мы решаем конкретный вопрос о его теме.

Иногда одного класса недостаточно. В письме «Не могу оплатить общежитие через портал» есть и вопрос об общежитии, и техническая проблема. Система с одной меткой должна выбрать основной маршрут. Система с несколькими метками может отметить обе темы. Какой вариант нужен, зависит от того, как сотрудники разбирают обращения. Число выходных меток — часть постановки, а не свойство самого русского языка.

В извлечении информации программа указывает части текста и их типы. Из сообщения «Запишите меня на консультацию 12 октября» можно выделить дату. Если бы рядом стояло имя преподавателя, можно было бы выделить и его. Здесь важно не просто узнать тему, но и правильно определить границы нужного фрагмента.

В поиске выходом становится упорядоченный список. На вопрос «Как продлить книгу?» система должна поднять инструкцию о продлении выше документа о заселении. Список из сотни документов мало помогает, если нужный ответ оказался в самом конце. Поэтому для поиска важно положение полезного результата, а не только его присутствие где-то в выдаче.

В генерации программа создаёт новый текст. Это может быть перевод, краткое содержание письма или ответ на вопрос. Теперь возможны ошибки, которые не укладываются в схему «выбран неправильный класс»: потерянное условие,

неподходящий стиль, лишняя подробность, неверное число. Мы вернёмся к ним, когда будем работать с языковыми моделями.

Сравним эти выходы на одном примере.

Задача	Возможный результат для «Не могу войти в личный кабинет»
Классификация	Категория «ИТ»
Извлечение	Упоминание сервиса «личный кабинет»
Поиск	«Восстановление доступа» — первый документ в выдаче
Генерация	Уточняющий вопрос «Что появляется при входе?»

Такое сравнение помогает избежать распространённой ошибки: выбрать знакомую модель, а потом подстраивать под неё задачу. Нам удобнее двигаться в обратном порядке. Сначала определим полезный результат, затем подберём данные и только после этого способ его получить.

Почему отдельных слов недостаточно

Слайд 4. Контекст выбирает значение

Рассмотрим слово «ключ». В сочетании «ключ от комнаты» оно связано с замком. В сочетании «ключ доступа к сервису» — с разрешением работать с системой. Если программа видит только само слово, этих двух значений она не различает. Соседние слова дают информацию, которой нет в отдельной записи словаря.

Неоднозначным бывает и устройство предложения. «Я видел преподавателя с телескопом» допускает два прочтения: телескоп мог быть у преподавателя, а мог использоваться для наблюдения. Чтобы выбрать одно, хочется знать ситуацию. Можно представить сцену у обсерватории, где человек смотрит в телескоп, или сцену, где преподаватель несёт прибор. Набор слов одинаков, отношения между участниками различаются.

Слайд 5. Одно слово может обмануть

Теперь фраза «Отлично, портал снова не работает». Если искать только положительные и отрицательные слова, «отлично» будет толкать решение в сторону положительной оценки. Но вместе с остальной фразой оно, скорее всего, выражает раздражение. Здесь приходится учитывать сочетание слов и способ употребления, а не просто складывать их словарные характеристики.

Обратите внимание, что контекст нужен в каждом примере по-своему. Для «ключа» помогают соседние существительные. Для телескопа может понадобиться описание ситуации. Для иронии важны ожидания говорящего. Мы не обязаны начинать курс с

системы, которая одинаково хорошо решает все эти случаи. Но должны понимать, какую информацию теряем, когда упрощаем представление текста.

Слайд 6. Набор лемм теряет роли

Русский язык даёт наглядный пример такого упрощения. Сравните «Преподаватель консультирует студента» и «Студент консультирует преподавателя». Если привести слова к начальной форме и оставить только их набор, оба предложения дадут «преподаватель», «консультировать», «студент». Однако роли участников поменялись. Порядок слов и окончания помогали различать эти роли; набор лемм их не сохранил.

Для определения общей темы этот набор может оказаться достаточным: оба сообщения о консультации. Для вопроса «Кто кого консультирует?» — уже нет. Так мы получаем важное практическое правило: оценивать обработку текста нужно относительно задачи. Одна и та же операция может помочь искать похожие темы и одновременно помешать разбирать отношения между участниками.

Повседневная переписка добавляет сокращения и неточные формулировки. «Не могу авторизоваться», «не пускает в лк» и «вход не работает» могут описывать одну проблему. При этом «вход в корпус закрыт» относится уже к зданию. Позже мы сравним способы представлять такие тексты: от подсчёта совпадающих фрагментов до моделей, которые учитывают окружение слова.

Из сообщений получается корпус

Слайд 7. Корпус хранит связи

В Python отдельный текст обычно хранится в строке. Для обучения и проверки модели нужна коллекция текстов — корпус. У каждого текста должны быть понятное место в этой коллекции и связь с другими сведениями: категорией, датой, источником или диалогом, из которого он взят.

В наших лабораторных корпус обращений устроен как таблица. `id` обозначает конкретную запись, `text` содержит сообщение, `label` — его тему. Поле `group_id` связывает разные формулировки одного обращения. В `split` указано, к какой части выборки относится запись; подробно с разделением данных мы познакомимся на следующей лекции.

Слайд 8. 600 строк — 120 ситуаций

Почему недостаточно просто посчитать строки? Возьмём исходное обращение «Не работает учебный портал» и четыре варианта, как в нашем корпусе: с приветствием «Здравствуйте!», с просьбой «Подскажите, пожалуйста.», с благодарностью «Буду благодарен за ответ.» и с фразой «Нужна консультация.». В таблице пять строк. Но

это не пять независимо возникших проблем — мы пять раз описали одну ситуацию. На схеме эти пять записей обозначены как основа, приветствие, просьба, благодарность и консультация. Если часть вариантов окажется среди примеров для обучения, а часть — среди проверочных, модель встретит на проверке очень знакомые сообщения.

Поэтому мы считаем и строки, и исходные ситуации. В учебном корпусе 600 строк образованы из 120 базовых обращений, по пять вариантов на каждое. Эти числа описывают состав набора. При проверке переноса на новые ситуации нам будет важно разделять целые группы, чтобы перефразировки одного обращения оставались вместе.

Слайд 9. Ошибки видны до обучения

Теперь посмотрим на три подозрительные записи.

Текст	Категория
«Не открывается учебный портал·»	ИТ
Строка из пробелов	Библиотека
«Когда можно продлить книгу?»	Общежитие

Точка «·» в первом примере обозначает пробел в конце. Если в корпусе есть обращение «Не открывается учебный портал» без этого пробела, после нормализации строки совпадут. Второй записи не хватает содержания, а третья вызывает вопрос к метке. Все три проблемы можно заметить ещё до обучения. Но решения будут разными: для повтора нужно выяснить происхождение записей, для пустого текста — причину его появления, для категории — прочитать сообщение и правила разметки.

В реальных данных одинаковые сообщения иногда отправляют разные люди. Тогда совпадение текста ещё не означает, что одна строка лишняя. Например, пять обращений «Не работает портал» могут отражать массовый сбой. Если мы оцениваем число поступивших жалоб, удалять четыре из них нельзя. Если строим проверку на новых формулировках, такие совпадения, наоборот, нужно учитывать при разделении. Назначение корпуса определяет, что делать с повтором.

Пустота тоже имеет несколько представлений. В таблице может быть пропуск, пустая строка "" или строка из пробелов. Проверка только на одно из них пропустит остальные. Сначала полезно посмотреть на типы и значения, а затем сформулировать правило. В наборе для первой лабораторной пустой считается строка, в которой после удаления крайних пробелов не осталось символов.

Метка выражает принятое решение

Слайд 10. Метка следует правилу

Вернёмся к письму «Не могу оплатить общежитие через портал». Какую категорию ему поставить? Если студента направляют по теме платежа, можно выбрать общежитие. Если сначала нужно восстановить работу сайта, можно выбрать ИТ. До чтения правил оба решения выглядят объяснимыми.

Разметка появляется из договорённости о задаче. В инструкции можно, например, указать: вопросы о необходимых документах и порядке заселения направлять в общежитие; сообщения о сбое интерфейса, пароле и коде ошибки — в ИТ. Тогда для нашего обращения хочется уточнить, что именно мешает оплате. Если сообщение содержит «появляется ошибка 403», аргумент в пользу технического маршрута становится сильнее.

Полезная инструкция состоит не только из названий классов. В ней нужны примеры, границы категорий и правило для спорных случаев. Иначе два разметчика могут последовательно следовать разным представлениям о задаче, а в таблице это будет выглядеть как случайные ошибки.

Попробуйте распределить три обращения: «Где взять договор проживания?», «Не открывается форма оплаты общежития», «Какие документы нужны для социальной стипендии?». Для первого и третьего тема достаточно ясна. Во втором присутствует слово «общежитие», но само затруднение связано с формой на сайте. Такой пример полезнее одного очевидного вопроса о книге: он проверяет границу между категориями.

Слайд 11. Конфликт меток требует решения

В таблице могут встретиться одинаковые тексты с разными метками. Возьмём две записи «Когда можно продлить книгу?». У одной метка «Библиотека», у другой — «Общежитие». Мы рассматриваем задачу, где каждому обращению назначают одну категорию, поэтому такое расхождение нужно проверить. После нормализации обнаружить это просто: сгруппировать строки по тексту и посчитать число различных категорий. Однако программа ещё не знает, какая категория верна. Она нашла место, где требуется решение. Если оставить первую попавшуюся строку, конфликт исчезнет из таблицы, но причина расхождения останется неизвестной.

Бывает и обратная ситуация: никакого точного повтора нет, а метка всё равно выглядит неверной. Вопрос о продлении книги попал в общежитие — такое замечают при чтении примеров. Поэтому автоматические проверки и выборочный просмотр дополняют друг друга. Одна проверка на дубликаты не заменяет знакомства с корпусом.

Есть короткие обращения, для которых самого текста недостаточно. «Можно ли отправить пакет документов по почте?» в нашем наборе относится к поступлению, потому что так была задумана исходная ситуация. Но без этой информации фраза могла бы касаться и другой процедуры. На таких случаях удобно обсуждать, какие сведения стоит добавить во вход, когда задать уточняющий вопрос и как оценивать спорный ответ модели.

Что считать токеном

Слайд 12. Границы токена задаёт метод

Прежде чем представить текст числами, обычно нужно выделить элементы, с которыми будет работать программа. Такие элементы называют токенами. Токен может совпадать со словом, быть знаком пунктуации или частью слова. Его границы определяются выбранным способом разбиения.

Возьмём строку «ИТ-поддержка: вход не работает!». Метод `split()` разделит её по пробелам и вернёт четыре элемента:

```
ИТ-поддержка: | вход | не | работает!
```

Дефис, двоеточие и восклицательный знак остались рядом со словами. Теперь применим правило, которое отдельно выделяет последовательности букв, цифр и подчёркиваний, а также знаки. В Python его можно записать так:

```
re.findall(r"\w+|[^\w\s]", text)
```

Получится восемь элементов:

```
ИТ | - | поддержка | : | вход | не | работает | !
```

Количество изменилось, хотя сообщение осталось прежним. Поэтому, встретив утверждение «текст содержит сто токенов», сначала стоит узнать, каким токенизатором его разбили. Это особенно важно позже, когда длина входа модели будет ограничена числом токенов.

Слайд 13. Адрес распался на части

Рассмотрим адрес `help@sever.example`. Для человека это один адрес электронной почты. Наше регулярное выражение выделит `help`, `@`, `sever`, `.`, `example`. Если задача — найти адрес целиком, понадобится другое правило или дополнительный этап объединения. Если задача — собрать статистику фрагментов текста, такое разбиение может быть удобным.

Слайд 14. Повтор не расширяет словарь

Ещё одно различие — между вхождением и типом токена. В строке «книга книга журнал» три вхождения, но только два разных элемента. Список сохраняет оба появления слова «книга», множество — только одно. На следующей лекции число появлений станет одним из признаков текста, поэтому эти две величины нельзя подменять друг другом.

Предобученные модели часто используют подсловную токенизацию. Редкое слово может разбиваться на несколько частей, которые уже есть в словаре. Такое разбиение не обязано совпадать с морфемами или школьным разбором слова. Когда мы дойдём до BERT и генеративной модели, будем использовать их собственные токенизаторы и смотреть, какие элементы они действительно создают.

Что можно менять при нормализации

Слайд 15. Какие различия можно убрать?

Нормализация приводит разные записи к выбранному общему виду. Самый простой пример — лишние пробелы. В строке `"  Не  отменяйте запись!  "` два пробела между словами и пробелы по краям не добавляют полезной информации для маршрутизации обращения. Пробелы по краям удаляем, а повторные пробелы между словами заменяем одним. Получается `"Не отменяйте запись!"`. На слайде пробелы отмечены точками «·», чтобы изменение было видно; в самой строке никаких точек мы не добавляем.

На Python это небольшая функция. Но за ней всё равно стоит решение о том, какие различия нам не нужны. В обычном обращении сжатие пробелов удобно. В программном коде или таблице пробелы могут участвовать в структуре. Поэтому функцию, подготовленную для сообщений, не стоит без изменений применять к любому текстовому файлу.

Часто текст также переводят в нижний регистр. Для тематического поиска «Книга» и «книга» полезно считать одним словом. Но в других задачах заглавная буква помогает заметить имя или аббревиатуру. Приведение к нижнему регистру — ещё одно решение, которое стоит проверять на характерных примерах своей задачи.

Слайд 16. Один вид — две записи

Unicode даёт другой пример внешне одинаковых записей. Буква «й» может быть представлена одним символом `U+0439` или последовательностью «и» (`U+0438`) и комбинируемого знака краткости (`U+0306`). Форма NFC позволяет согласовать такие канонически эквивалентные записи. При этом похожие латинская `a` и кириллическая `а` остаются разными буквами: нормализация Unicode не исправляет смешение алфавитов.

Слайд 17. Удаление «не» меняет смысл

Самые наглядные потери возникают при удалении служебных слов. Сравните «Я доволен работой портала» и «Я не доволен работой портала». Если убрать «я» и «не», останется одна и та же строка. Модель, получающая только эту строку, уже не сможет восстановить различие между двумя отзывами: оно исчезло до её вызова.

Слайд 18. Почему важен номер ошибки

С числами происходит похожее. После удаления цифр «Ошибка 403» превращается в «Ошибка». Чтобы определить, что обращение связано с технической проблемой, этого может хватить. Для выбора инструкции по устранению сбоя номер важен. Если заранее не определить задачу, легко принять потерю полезного признака за удачную очистку.

Удобно составить небольшую таблицу решений: операция, пример до и после, что стало одинаковым, что потерялось. В неё можно включить пробелы, регистр, отрицание, цифры и пунктуацию. Для каждой строки попробуйте назвать задачу, где преобразование помогает, и задачу, где оно мешает. Так разговор о предобработке становится предметным.

Слайд 19. Повторная обработка стабильна

Ещё одно полезное свойство — повторное применение нормализации не должно менять уже нормализованный текст. Это называют идемпотентностью: `f(f(x)) = f(x)`. Возьмём строку `"  Ошибка  403  "`. Здесь `f` удаляет пробелы по краям и сжимает повторные пробелы внутри строки. После первого применения получаем `"Ошибка 403"`, после второго — ровно ту же строку. На схеме два шага позволяют сравнить результаты непосредственно. Такое свойство удобно проверять в коде. Оно говорит о стабильности функции, но само по себе не отвечает на вопрос о сохранении смысла: стабильно удалять отрицание тоже возможно.

Словоформа, лемма и основа

Слайд 20. Лемма связывает словоформы

«Книга», «книги», «книгой» — разные формы одного слова. Для поиска по теме полезно связать их с леммой «книга». Тогда запрос в одной форме сможет совпасть с документом, где стоит другая. Процедуру приведения словоформ к лемме называют лемматизацией.

От лемматизации отличается стемминг: правило отсекает части слова и оставляет основу для сопоставления. Результат может не быть словарным словом. Это допустимо, если он используется как признак, но его не следует выдавать за

начальную форму русского слова. Названия процедур отражают разные цели и способы работы.

Слайд 21. Одна форма — разные леммы

Но форма не всегда однозначно указывает на лемму. В сочетании «образцы стали» речь идёт о материале, и подходящая лемма — «сталь». В предложении «студенты стали проверять результаты» используется глагол «стать». Если словарь вернул оба варианта, он помог обнаружить неоднозначность, но выбор ещё предстоит сделать по окружению.

Возвращаясь к предложениям о преподавателе и студенте, можно увидеть цену сведения к начальной форме. Леммы объединяют словоформы, но сами по себе не сохраняют падеж. Если после этого ещё убрать порядок, различие ролей исчезнет. Для тематической классификации такое представление иногда удобно; для извлечения отношений нужны дополнительные сведения.

Стоит ли обязательно лемматизировать текст перед любой моделью? У разных представлений разная чувствительность к словоформам. Мешок слов считает формы разными признаками. Представление через символьные фрагменты позволяет учитывать общие части разных словоформ. Предобученная модель уже имеет свой способ токенизации и представления. В дальнейшем мы будем выбирать обработку вместе с моделью, а не составлять один обязательный список очистки на весь курс.

Для первой лабораторной достаточно маленького словаря кандидатов, чтобы увидеть проблему на нескольких словах. Это позволит проследить решение вручную. При работе с настоящим морфологическим анализатором логика вопроса останется прежней: какие варианты он предложил, какой контекст доступен и почему выбран именно этот вариант.

Как организовать подготовку данных

Слайд 22. Сохраняйте исходную строку

Соберём разобранные операции в последовательность. Сначала читаем таблицу и проверяем её устройство: есть ли нужные поля, какие у них типы, сколько записей удалось загрузить. Затем ищем пустые сообщения и неизвестные категории. После этого создаём нормализованный столбец и сравниваем его с исходным.

Порядок здесь имеет значение. Если искать повторы до обработки пробелов, две одинаковые по содержанию строки могут показаться разными. Если сначала удалить пустые записи и забыть сохранить их список, будет трудно объяснить, почему число строк уменьшилось. Для каждого преобразования полезно понимать, какие данные поступили на вход и какие получились на выходе.

Исходный текст оставляем в таблице. Это самый простой способ выяснить, откуда появилась странность в обработанном результате. Проследим три представления одного обращения. В `text` хранится исходная строка `"  Не  отменяйте запись!  "`. В `normalized` — `"Не отменяйте запись!"`. После разбиения нашим регулярным выражением в `tokens` будут `Не`, `отменяйте`, `запись`, `!`. По исходной строке видно, что мы изменили пробелы, сохранили отрицание и выделили знак в отдельный токен. Сравнение помогает установить, на каком шаге появилось каждое изменение.

Проверки лучше писать рядом с понятными примерами. Для нормализации это строка с лишними пробелами, пустая строка, составная Unicode-запись и сообщение с важным отрицанием. Для токенизатора — дефис, пунктуация, адрес и повтор слова. Для разметки — неизвестная категория и одинаковые тексты с разными метками.

Хороший контрольный пример изолирует одно свойство. Если мы хотим проверить сохранение номера ошибки, достаточно строки «Ошибка 403». Если одновременно добавить приветствие, адрес, переносы и несколько опечаток, при неправильном результате будет труднее понять причину. Сложные примеры нужны тоже, но начинать отладку удобнее с небольших.

Представьте, что функция сначала переводит всё в нижний регистр, а затем другая часть программы пытается найти аббревиатуру «ИТ» по заглавным буквам. Каждая операция по отдельности может выглядеть разумно, но вместе они конфликтуют. Подготовка данных — это последовательность связанных решений. Проверять только последнюю строку без знания предыдущих шагов недостаточно для отладки.

Позже к этой последовательности добавятся построение признаков, обучение и предсказание. Некоторые шаги будут запоминать сведения из обучающих данных: словарь, частоты слов, параметры модели. На следующем занятии разберём, почему таким шагам нельзя заранее показывать проверочную выборку. Сейчас важно освоить более простую привычку: сохранять вход, понимать действие каждого шага и уметь показать его результат на конкретной строке.

С чего начнётся лабораторная

Слайд 23. Что нужно сохранить для задачи?

Проверим понимание на новой строке:

```
"  Не  отменяйте запись на 12 октября!  "
```

Что уберёт минимальная нормализация? Что должно остаться? А если будущая задача — извлечь дату, какие части сообщения понадобятся программе? Ответы на эти вопросы связывают обработку текста с тем результатом, который мы хотим получить от системы.

В первой работе вы получите основную таблицу обращений и небольшой файл для аудита. Начните с чтения нескольких строк. После этого посчитайте записи и группы, найдите пустые тексты и неизвестные метки. Напишите минимальную нормализацию, сравните тексты после неё и разберите обнаруженные совпадения.

Затем два токенизатора дадут разные разбиения одних и тех же сообщений. Нас будет интересовать не только число элементов, но и их назначение: сохранился ли адрес, что стало с дефисом, чем повтор слова отличается от нового слова в словаре. В последнем опыте агрессивная обработка сделает два противоположных отзыва одинаковыми. Объяснить этот результат важнее, чем просто получить совпадение двух строк в проверке.

К концу работы у вас должна получиться небольшая история о данных: какие проблемы были найдены, какие изменения внесены в обработанную версию и почему. В выводах используйте сами обращения. Формулировка «при удалении слова “не” два противоположных отзыва стали одинаковыми» сообщает больше, чем общая фраза «предобработка влияет на качество».

На следующем занятии из подготовленных сообщений получится матрица признаков. Мы обучим простой классификатор и узнаем, насколько хорошо он направляет обращения по темам. Но сначала у нас появится понятный корпус — с исходными строками, объяснимой разметкой и проверенной обработкой.

Для дальнейшего чтения

– Jurafsky, Martin. Speech and Language Processing — задачи обработки языка, слова и токенизация.

– Dive into Deep Learning: Text Preprocessing — последовательный разбор текста, токенов и словаря.

– Документация Python: unicodedata.normalize — формы нормализации Unicode.

Обращения и правила университета «Север» созданы для учебного курса. Описание набора находится в папке `data`.